Layer-wise Conditioning Analysis in Exploring the Learning Dynamics of DNNs

Lei Huang[1] Jie Qin[1] Li Liu[1] Fan Zhu[1] Ling Shao[2,1]

[1]Inception Institute of Artificial Intelligence (IIAI), Abu Dhabi, UAE
[2]Mohamed bin Zayed University of Artificial Intelligence, Abu Dhabi, UAE
{lei.huang, jie.qin, li.liu, fan.zhu, ling.shao}@inceptioniai.org

Abstract. Conditioning analysis uncovers the landscape of an optimization objective by exploring the spectrum of its curvature matrix. This has been well explored theoretically for linear models. We extend this analysis to deep neural networks (DNNs) in order to investigate their learning dynamics. To this end, we propose layer-wise conditioning analysis, which explores the optimization landscape with respect to each layer independently. Such an analysis is theoretically supported under mild assumptions that approximately hold in practice. Based on our analysis, we show that batch normalization (BN) can stabilize the training, but sometimes result in the false impression of a local minimum, which has detrimental effects on the learning. Besides, we experimentally observe that BN can improve the layer-wise conditioning of the optimization problem. Finally, we find that the last linear layer of a very deep residual network displays ill-conditioned behavior. We solve this problem by only adding one BN layer before the last linear layer, which achieves improved performance over the original and pre-activation residual networks.

Keywords: Conditioing Analysis; Normalization; Residual Network

1 Introduction

Deep neural networks (DNNs) have been extensively used in various domains [26]. Their success depends heavily on the improvement of training techniques [17,22,15], *e.g.*, fine weight initialization [17,12,39,14], normalization of internal representations [22,46], and well-designed optimization methods [49,24]. It is believed that these techniques are well connected to the curvature of the loss [39,38,25]. Analyzing this curvature is thus essential in determining various learning behaviors of DNNs.

In the interest of optimization, conditioning analysis uncovers the landscape of an optimization objective by exploring the spectrum of its curvature matrix. This has been well explored for linear models both in terms of regression [28] and classification [44], where the convergence condition of the optimization problem is controlled by the maximum eigenvalue of the curvature matrix [28,27], and the learning time of the model is lower-bounded by its condition number [28,27]. However, in the context of deep learning, the conditioning analysis suffers from

several barriers: 1) the model is over-parameterized and whether the direction with respect to small/zero eigenvalues contributes to the optimization progress is unclear [37,34]; 2) the memory and computational costs are extremely high [37,11].

This paper aims to bridge the gap between the theoretical analyses developed by the optimization community and the empirical techniques used for training DNNs, in order to better understand the learning dynamics of DNNs. We propose a layer-wise conditioning analysis, where we analyze the optimization landscape with respect to each layer independently by exploring the spectra of their curvature matrices. The motivation behind our layer-wise conditioning analysis is based on the recent success of second curvature approximation techniques in DNNs [32,31,1,41,3]. We show that the maximum eigenvalue and the condition number of the block-wise Fisher information matrix (FIM) can be characterized based on the spectrum of the covariance matrix of the input and output-gradient, under mild assumptions, which makes evaluating optimization behavior practical in DNNs. Another theoretical base is the recently proposed proximal back-propagation [7,10,50] where the original optimization problem can be approximately decomposed into multiple independent sub-problems with respect to each layer [50]. We provide the connection between our analysis and the proximal back-propagation [10].

Based on our layer-wise conditioning analysis, we show that batch normalization (BN) [22] can adjust the magnitude of the layer activations/gradients, and thus stabilizes the training. However, this kind of stabilization can drive certain layers into a particular state, referred to as **weight domination**, where the gradient update is feeble. This sometimes has detrimental effects on the learning (Section 4.1). We also experimentally observe that BN can improve the layer-wise conditioning of the optimization problem. Furthermore, we find that the unnormalized network has several small eigenvalues in the layer curvature matrix, which are mainly caused by the so-called **dying neurons** (Section 4.2), while this behavior is almost entirely absent in batch normalized networks.

We further analyze the ignored difficulty in training very deep residual networks [15]. Using our layer-wise conditioning analysis, we show that the difficulty mainly arises from the ill-conditioned behavior of the last linear layer. We solve this problem by only adding one BN layer before the last linear layer, which achieves improved performance over the original [15] and pre-activation [16] residual networks (Section 5).

2 Preliminaries

Optimization Objective Consider a true data distribution $p_*(\mathbf{x}, \mathbf{y}) = p(\mathbf{x})p(\mathbf{y}|\mathbf{x})$ and the sampled training sets $\mathcal{D} \sim p_*(\mathbf{x}, \mathbf{y})$ of size N. We focus on a supervised learning task aiming to learn the conditional distribution $p(\mathbf{y}|\mathbf{x})$ using the model $q(\mathbf{y}|\mathbf{x})$, where $q(\mathbf{y}|\mathbf{x})$ is represented as a function $f_\theta(\mathbf{x})$ parameterized by θ. From an optimization perspective, we aim to minimize the empirical risk, averaged over the sample loss represented as $\ell(\mathbf{y}, f_\theta(\mathbf{x}))$ in training sets $\mathcal{D}$: $\mathcal{L}(\theta) = \frac{1}{N}\sum_{i=1}^{N}(\ell(\mathbf{y}^{(i)}, f_\theta(\mathbf{x}^{(i)})))$.

Gradient Descent In general, the gradient descent (GD) update seeks to iteratively reduce the loss $\mathcal{L}$ by $\theta_{t+1} = \theta_t - \eta \frac{\partial \mathcal{L}}{\partial \theta}$, where η is the learning rate. For large-scale learning, stochastic gradient descent (SGD) is extensively used to approximate the gradients $\frac{\partial \mathcal{L}}{\partial \theta}$ with a mini-batch gradient. In theory, the convergence behaviors (*e.g.*, the number of iterations required for convergence to a stationary point) depend on the Lipschitz constant C_L of the gradient function of $\mathcal{L}$, which characterizes the global smoothness of the optimization landscape. In practice, the Lipschitz constant is either unknown for complicated functions or too conservative to characterize the convergence behaviors [5]. Researchers thus turn to the local smoothness, characterized by the Hessian matrix $\mathbf{H} = \frac{\partial \mathcal{L}^2}{\partial \theta \partial \theta}$ under the condition that $\mathcal{L}$ is twice differentiable.

Approximate Curvature Matrices The Hessian describes the local curvature of the optimization landscape. Such curvature information intuitively guides the design of second-order optimization algorithms [37,5], where the update direction is adjusted by multiplying the inverse of a pre-conditioned matrix $\mathbf{G}$ as: $\frac{\partial \widehat{\mathcal{L}}}{\partial \theta} = \mathbf{G}^{-1} \frac{\partial \mathcal{L}}{\partial \theta}$. $\mathbf{G}$ is a positive definite matrix that approximates the Hessian and is expect to sustain the its positive curvature. The second moment matrix of sample gradient: $\mathbf{M} = \mathbb{E}_{\mathcal{D}}(\frac{\partial \ell}{\partial \theta} \frac{\partial \ell}{\partial \theta}^T)$ is usually used as the pre-conditioned matrix [36,29]. Besides, Pascanu and Bengio [35] showed that the FIM: $\mathbf{F} = \mathbb{E}_{p(\mathbf{x}),\ q(\mathbf{y}|\mathbf{x})}(\frac{\partial \ell}{\partial \theta} \frac{\partial \ell}{\partial \theta}^T)$ can be viewed as a pre-conditioned matrix when performing the natural gradient descent algorithm [35]. Fore more analyses on the connections among $\mathbf{H}$, $\mathbf{F}$, $\mathbf{M}$ please refer to [30,5]. In this paper, we refer to the analysis of the spectrum of the (approximate) curvature matrices as *conditioning analysis*.

Conditioning Analysis for Linear Models Consider a linear regression model with a scalar output $f_{\mathbf{w}}(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$, and mean square error loss $\ell = (y - f_\theta(\mathbf{x}))^2$. As shown in [28,27], the learning dynamics in such a quadratic surface are fully controlled by the spectrum of the Hessian matrix $\mathbf{H} = \mathbb{E}_{\mathcal{D}}(\mathbf{x}\mathbf{x}^T)$. There are two statistical momentums that are essential for evaluating the convergence behaviors of the optimization problem. One is the maximum eigenvalue of the curvature matrix λ_{max}, and the other is the condition number of the curvature matrix, denoted by $\kappa = \frac{\lambda_{max}}{\lambda_{min}}$, where λ_{min} is the minimum nonzero eigenvalue of the curvature matrix. Specifically, λ_{max} controls the upper bound and the optimal learning rate (*e.g.*, the optimal learning rate is $\eta = \frac{1}{\lambda_{max}(\mathbf{H})}$ and the training will diverge if $\eta \geq \frac{2}{\lambda_{max}(\mathbf{H})}$). κ controls the iterations required for convergence (*e.g.*, the lower bound of the iteration is $\kappa(\mathbf{H})$ [28]). If $\mathbf{H}$ is an identity matrix that can be obtained by whitening the input, the GD can converge within only one iteration. It is easy to extend the solution of linear regression from a scalar output to a vectorial output $f_{\mathbf{W}}(\mathbf{x}) = \mathbf{W}^T\mathbf{x}$. In this case, the Hessian is represented as

$$\mathbf{H} = \mathbb{E}_{\mathcal{D}}(\mathbf{x}\mathbf{x}^T) \otimes \mathbf{I}, \tag{1}$$

where $\otimes$ indicates the Kronecker product [13] and $\mathbf{I}$ denotes the identity matrix. For the linear classification model with cross entropy loss, the Hessian is approximated by [44]:

$$\mathbf{H} = \mathbb{E}_{\mathcal{D}}(\mathbf{x}\mathbf{x}^T) \otimes \mathbf{S}. \tag{2}$$

$\mathbf{S} \in \mathbb{R}^{c\times c}$ is defined by $\mathbf{S} = \frac{1}{c}(\mathbf{I}_c - \frac{1}{c}\mathbf{1}_c)$, where c is the number of classes and $\mathbf{1}_c \in \mathbb{R}^{c\times c}$ denotes a matrix in which all entries are 1. Eqn. 2 assumes the Hessian does not significantly change from the initial region to the optimal region [44].

3 Layer-wise Conditioning Analysis for DNNs

Considering a multilayer perceptron (MLP), $f_\theta(\mathbf{x})$ can be represented as a layer-wise linear and nonlinear transformation, as follows:

$$\mathbf{h}_k = \mathbf{W}_k \mathbf{x}_{k-1}, \quad \mathbf{x}_k = \phi(\mathbf{h}_k), \quad k = 1, ..., K, \tag{3}$$

where $\mathbf{x}_0 = \mathbf{x}$ and the learnable parameters $\theta = \{\mathbf{W}_k \in \mathbb{R}^{d_k \times d_{k-1}}, k = 1, ..., K\}$. To simplify the denotation, we set $\mathbf{x}_K = \mathbf{h}_K$ as the output of the network $f_\theta(\mathbf{x})$.

A conditioning analysis on the full curvature matrix for DNNs is difficult due to the high memory and computational costs [11,34]. We thus seek to analyze an approximation of the curvature matrix. One successful example in second-order optimization over DNNs is approximating the FIM using the Kronecker product (K-FAC) [31,1,41,3]. In the K-FAC approach, there are two assumptions: 1) weight-gradients in different layers are assumed to be uncorrelated; 2) the input and output-gradient in each layer are approximated as independent, so the full FIM can be represented as a block diagonal matrix, $\mathbf{F} = diag(F_1, ..., F_K)$, where F_k is the sub-FIM (the FIM with respect to the parameters in a certain layer) and computed as:

$$F_k = \mathbb{E}_{p(\mathbf{x}),\ q(\mathbf{y}|\mathbf{x})}((\mathbf{x}_k\mathbf{x}_k^T) \otimes (\frac{\partial \ell}{\partial \mathbf{h}_k}^T \frac{\partial \ell}{\partial \mathbf{h}_k})) \approx \mathbb{E}_{p(\mathbf{x})}(\mathbf{x}_k\mathbf{x}_k^T) \otimes \mathbb{E}_{q(\mathbf{y}|\mathbf{x})}(\frac{\partial \ell}{\partial \mathbf{h}_k}^T \frac{\partial \ell}{\partial \mathbf{h}_k}). \tag{4}$$

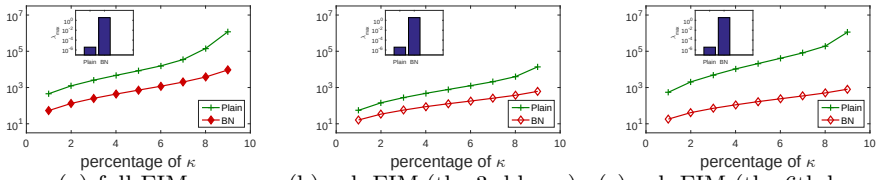
$\mathbf{x}_k$ denotes the layer input, and $\frac{\partial \ell}{\partial \mathbf{h}_k}$ denotes the layer output-gradient. We note that Eqn. 4 is similar to Eqn. 1 and 2, and all of them depend on the covariance matrix of the (layer) input. The main difference is that, in Eqn. 4, the covariance of output-gradient is considered and its value changes over different optimization regions, while in Eqn. 1 and 2, the covariance of output-gradient is constant.

Based on this observation, we propose layer-wise conditioning analysis, *i.e.*, we analyze each sub-FIM F_k's spectrum $\lambda(F_k)$ independently. We expect the spectra of sub-FIMs: $\{\lambda(F_k)\}_{k=1}^K$ to effectively reveal that of the full FIM: $\lambda(\mathbf{F})$, at least in terms of analyzing the learning dynamics of the DNNs. Specifically, we analyze the maximum eigenvalue $\lambda_{max}(F_k)$ and condition number $\kappa(F_k)$[1]. Based on the conclusion on the conditioning analysis of linear models shown in Section 2, there are two remarkable properties that can be used to implicitly uncover the landscape of the optimization problem:

- *Property 1*: $\lambda_{max}(F_k)$ indicates the magnitude of the weight-gradient in each layer, which shows the steepness of the landscape w.r.t.different layers.
- *Property 2*: $\kappa(F_k)$ indicates how easy it is to optimize the corresponding layer.

Discussion One concern is the validity of the assumptions the K-FAC approximation is based on. Note that [30,31] have provided some empirical evidence to

[1] We evaluate the general condition number with respect to the percentage: $\kappa_p = \frac{\lambda_{max}}{\lambda_p}$, where λ_p is the pd-th eigenvalue (in descending order) and d is the number of eigenvalues, *e.g.*, $\kappa_{100\%}$ is the original definition of the condition number.

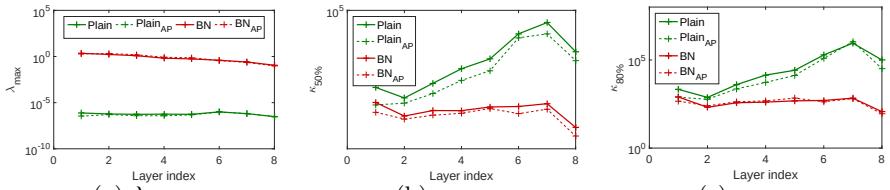


(a) full FIM (b) sub-FIM (the 3rd layer) (c) sub-FIM (the 6th layer)

Fig. 1. Conditioning analysis for unnormalized ('Plain') and normalized networks ('BN'). We show the maximum eigenvalue λ_{max} and the generalized condition number κ_p for comparison between the full FIM $\mathbf{F}$ and sub-FIMs $\{F_k\}$. The experiments are performed on an 8-layer MLP with 24 neurons in each layer, for MNIST classification. The input image is center-cropped and resized to 12×12 to remove uninformative pixels. We report the corresponding spectrum at random initialization [27]. Here, we report the results of the 3rd and 6th layers in (b) and (c), respectively. We have similar observations for other layers (See *SM*).

support their effectiveness in approximating the full FIM with block diagonal sub-FIMs. [23,43] also exploited similar assumptions to derive the mean&variance of eigenvalues (and maximum eigenvalue) of the full FIM, which is calculated using information from layer inputs and output-gradients. Here, we argue that the assumptions required for our analysis are weaker than those of the K-FAC approximation, since we only care about whether or not the spectra of sub-FIMs can accurately reveal the spectrum of full FIM. We conduct experiments to analyze the training dynamics of the unnormalized ('Plain') and batch normalized [22] ('BN') networks, by looking at the spectra of full curvature matrix and sub-curvature matrices. Figure 1 shows the results based on an 8-layer MLP with 24 neurons in each layer. By observing the results from the full FIM (Figure 1 (a)), we find that: 1) the unnormalized network suffers from gradient vanishing (the maximum eigenvalue is around $1e^{-5}$), while the batch normalized network has an appropriate magnitude of gradient (the maximum eigenvalue is around 1); 2) 'BN' has better conditioning than 'Plain', which suggests batch normalization (BN) can improve the conditioning of the network, as observed in [38,11]. We also obtain a similar conclusion when observing the results from the sub-FIMs (Figure 1 (b), (c)). This experiment demonstrates that our layer-wise conditioning analysis has the potentiality to uncover the training dynamics of the networks if the full conditioning analysis can. We also conduct experiments on MLPs with different layers and neurons, and further analyze the spectrum of the second moment matrix of sample gradient $\mathbf{M}$ (please refer to the *Supplementary Materials (SM)* for details). We have the same observations as in the first experiment.

Furthermore, we find that investigating $\{\lambda(F_k)\}_{k=1}^{K}$ is more beneficial for diagnosing the problems behind training DNNs than investigating $\lambda(\mathbf{F})$, *e.g.*, it enables the gradient vanishing/explosion to be located with respect to a specific layer from $\{\lambda_{max}(F_k)\}_{k=1}^{K}$, but not $\lambda_{max}(\mathbf{F})$. For example, we know that the 8-layer unnormalized MLP described in Figure 1 suffers from difficulty in training, but we cannot accurately diagnose the problem by only investigating the spectrum of the full FIM. However, by looking into the layer inputs and output-gradients, we find that this MLP suffers from exponentially decreased



(a) λ_{max} (b) $\kappa_{50\%}$ (c) $\kappa_{80\%}$

Fig. 2. Validation in approximating the sub-FIMs. The experimental setups are the same as in Figure 1. We compare maximum eigenvalue λ_{max} and generalized condition number κ_p of the sub-FIMs (solid lines) and the approximated ones (dashed lines).

magnitudes of inputs (forward) and output-gradients (backward). This can be resolved this by using a better initialization with appropriate variance [14] or using BN [22]. We further elaborate on how to use the layer-wise conditioning analysis to 'debug' the training of DNNs in the subsequent sections.

3.1 Efficient Computation

We denote the covariance matrix of the layer input as $\mathcal{I}_{\mathbf{x}} = \mathbb{E}_{p(\mathbf{x})}(\mathbf{x}\mathbf{x}^T)$ and the covariance matrix of the layer output-gradient as $\mathcal{I}_{\nabla\mathbf{h}} = \mathbb{E}_{q(\mathbf{y}|\mathbf{x})}(\frac{\partial \ell}{\partial \mathbf{h}}^T \frac{\partial \ell}{\partial \mathbf{h}})$. The condition number and maximum eigenvalue of the sub-FIM F can be derived based on the spectrum of $\mathcal{I}_{\mathbf{x}}$ and $\mathcal{I}_{\nabla\mathbf{h}}$, as shown in the following proposition.

Proposition 1. *Given $\mathcal{I}_{\mathbf{x}}$, $\mathcal{I}_{\nabla\mathbf{h}}$ and $F = \mathcal{I}_{\mathbf{x}} \otimes \mathcal{I}_{\nabla\mathbf{h}}$, we have: 1) $\lambda_{max}(F) = \lambda_{max}(\mathcal{I}_{\mathbf{x}}) \cdot \lambda_{max}(\mathcal{I}_{\nabla\mathbf{h}})$; and 2) $\kappa(F) = \kappa(\mathcal{I}_{\mathbf{x}}) \cdot \kappa(\mathcal{I}_{\nabla\mathbf{h}})$.*

The proof is shown in the *SM*. Proposition 1 provides an efficient way to calculate the maximum eigenvalue and condition number of sub-FIM F by computing those of $\mathcal{I}_{\mathbf{x}}$ and $\mathcal{I}_{\nabla\mathbf{h}}$. In practice, we use the empirical distribution $\mathbb{D}$ to approximate the expected distribution $p(\mathbf{x})$ and $q(\mathbf{y}|\mathbf{x})$ when calculating $\mathcal{I}_{\mathbf{x}}$ and $\mathcal{I}_{\nabla\mathbf{h}}$, since this is very efficient and can be performed with only one forward and backward pass, as has been shown in FIM approximation [31,1].

Note that Proposition 1 depends on the second assumption of Eqn. 4. We experimentally demonstrate the effectiveness of such an approximation in Figure 2, finding that the maximum eigenvalue and the condition number of the sub-FIMs match well with the approximated ones.

3.2 Connection to Proximal Back-propagation

Carreira-Perpinan and Wang [7] proposed to use auxiliary coordinates to redefine the optimization object $\mathcal{L}(\theta)$ with equality constraints imposed on each neuron. They solved the constrained optimization by adding a quadratic penalty as:

$$\widetilde{\mathcal{L}}(\theta, \mathbf{z}) = \mathcal{L}(\mathbf{y}, f_K(\mathbf{W}_K, \mathbf{z}_{K-1})) + \sum_{k=1}^{K-1} \frac{\lambda}{2} \|\mathbf{z}_k - f_k(\mathbf{W}_k, \mathbf{z}_{k-1}))\|^2, \tag{5}$$

where $f_k(\cdot,\cdot)$ is a function with respect to each layer. As shown in [7], the solution for minimizing $\widetilde{\mathcal{L}}(\theta, \mathbf{z})$ converges to the solution for minimizing $\mathcal{L}(\theta)$ as $\lambda \to \infty$, under mild conditions. Furthermore, the proximal propagation [10] and the following back-matching propagation [50] reformulate each sub-problem independently

with a backward order, minimizing each layer object $\mathcal{L}_k(\mathbf{W}_k, \mathbf{z}_{k-1}; \hat{\mathbf{z}}_k)$, given the target signal $\hat{\mathbf{z}}_k$ from the upper layer, as follows:

$$\begin{cases} \mathcal{L}(\mathbf{y}, f_K(\mathbf{W}_K, \mathbf{z}_{K-1})), & for\ k = K \\ \frac{1}{2}\|\hat{\mathbf{z}}_k - f_k(\mathbf{W}_k, \mathbf{z}_{k-1}))\|^2, & for\ k = K-1, ..., 1. \end{cases} \tag{6}$$

It has been shown that the produced $\mathbf{W}_k$ using gradient update w.r.t. $\mathcal{L}(\theta)$ equals to the $\mathbf{W}_k$ produced by the back-matching propagation (Procedure 1 in [50]) with one-step gradient update w.r.t. Eqn. 6, given an appropriate step size. Note that the target signal $\hat{\mathbf{z}}_k$ is obtained by back-propagation, which means the loss $\mathcal{L}(\theta)$ would be smaller if $f_k(\mathbf{W}_k, \mathbf{z}_{k-1})$ is more close to $\hat{\mathbf{z}}_k$. The loss $\mathcal{L}(\theta)$ will be reduced more efficiently, if the sup-optimization problems in Eqn. 6 are well-conditioned. Please refer to [10,50] for more details. If we view the auxiliary variable as the pre-activation in a specific layer, the sub-optimization problem in each layer is formulated as:

$$\begin{cases} \mathcal{L}(\mathbf{y}, \mathbf{W}_K \mathbf{z}_{K-1}), & for\ k = K \\ \frac{1}{2}\|\hat{\mathbf{z}}_k - \mathbf{W}_k \mathbf{z}_{k-1}\|^2, & for\ k = K-1, ..., 1. \end{cases} \tag{7}$$

It is clear that the sub-optimization problems with respect to $\mathbf{W}_k$ are actually linear classification (for k=K) or regression (for $k = 1, ..., K-1$) models. Their conditioning analysis is thoroughly characterized in Section 2.

This connection suggests: 1) the quality (conditioning) of the full optimization problem $\mathcal{L}(\theta)$ is well correlated to its sub-optimization problems shown in Eqn. 7, whose local curvature matrix can be well explored; 2) We can diagnose the ill behaviors of a DNN by speculating its spectra with respect to certain layers.

4 Exploring Batch Normalized Networks

Let x denote the input for a given neuron in one layer of a DNN. Batch normalization (BN) [22] standardizes the neuron within m mini-batch data by:

$$BN(x^{(i)}) = \gamma \frac{x^{(i)} - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta, \tag{8}$$

where $\mu = \frac{1}{m}\sum_{i=1}^{m} x^{(i)}$ and $\sigma^2 = \frac{1}{m}\sum_{i=1}^{m}(x^{(i)} - \mu)^2$ are the mean and variance, respectively. The learnable parameters γ and β are used to recover the representation capacity. BN is a ubiquitously employed technique in various architectures [22,15,48,19] due to its ability in stabilizing and accelerating training. Here, we explore how BN stabilizes and accelerates training based on our layer-wise conditioning analysis.

4.1 Stabilizing Training

From the perspective of a practitioner, two phenomena relate to the instability in training a DNN: 1) the training loss first increases significantly and then diverges; or 2) the training loss hardly changes, compared to the initial condition. The former is mainly caused by weights with large updates (*e.g.*, exploded gradients or optimization with a large learning rate). The latter is caused by weights with few updates (vanished gradients or optimization with a small learning rate). In the following theorem, we show that the unnormalized rectifier neural network is very likely to encounter both phenomena.

Theorem 1. *Given a rectifier neural network (Eqn. 3) with nonlinearity $\phi(\alpha\mathbf{x}) = \alpha\phi(\mathbf{x})$ ($\alpha > 0$), if the weight in each layer is scaled by $\widehat{\mathbf{W}}_k = \alpha_k \mathbf{W}_k$ ($k = 1, ..., K$ and $\alpha_k > 0$), we have the scaled layer input: $\widehat{\mathbf{x}}_k = (\prod_{i=1}^{k} \alpha_i)\mathbf{x}_k$. Assuming that $\frac{\partial \mathcal{L}}{\partial \widehat{\mathbf{h}}_K} = \mu \frac{\partial \mathcal{L}}{\partial \mathbf{h}_K}$, we have the output-gradient: $\frac{\partial \mathcal{L}}{\partial \widehat{\mathbf{h}}_k} = \mu(\prod_{i=k+1}^{K} \alpha_i)\frac{\partial \mathcal{L}}{\partial \mathbf{h}_k}$, and weight-gradient: $\frac{\partial \mathcal{L}}{\partial \widehat{\mathbf{W}}_k} = (\mu \prod_{i=1, i\neq k}^{K} \alpha_i)\frac{\partial \mathcal{L}}{\partial \mathbf{W}_k}$, for all $k = 1, ..., K$.*

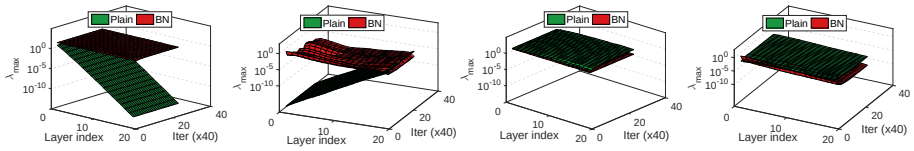
The proof is shown in the *SM*. From Theorem 1, we observe that the scaled factor α_k of the weight in layer k will affect all other layers' weight-gradients. Specifically, if all $\alpha_k > 1$ ($\alpha_k < 1$), the weight-gradient will increase (decrease) exponentially for one iteration. Moreover, such an exponentially increased weight-gradient will be sustained and amplified in the subsequent iteration, due to the increased magnitude of the weight caused by updating. That is why the unnormalized neural network will diverge, once the training loss increases over a few continuous iterations. We show that such instability can be relieved by BN, based on the following theorem.

Theorem 2. *Under the same condition as Theorem 1, for the normalized network with $\mathbf{h}_k = \mathbf{W}_k \mathbf{x}_{k-1}$ and $\mathbf{s}_k = BN(\mathbf{h}_k)$, we have: $\widehat{\mathbf{x}}_k = \mathbf{x}_k$, $\frac{\partial \mathcal{L}}{\partial \widehat{\mathbf{h}}_k} = \frac{1}{\alpha_k}\frac{\partial \mathcal{L}}{\partial \mathbf{h}_k}$, $\frac{\partial \mathcal{L}}{\partial \widehat{\mathbf{W}}_k} = \frac{1}{\alpha_k}\frac{\partial \mathcal{L}}{\partial \mathbf{W}_k}$, for all $k = 1, ..., K$.*

The proof is shown in the *SM*. From Theorem 2, the scaled factor α_k of the weight will not affect other layers' activations/gradients. The magnitude of the weight-gradient is inversely proportional to the scaled factor. Such a mechanism will stabilize the weight growth/reduction, as shown in [22,47]. Note that the behaviors when stabilizing training (Theorem 2) also apply for other activation normalization methods [2,45,18]. We note that the scale-invariance of BN in stabilizing training has been analyzed in previous work [2]. Different to their analyses on the normalization layer itself, we provide an explicit formulation of weight-gradients and output-gradients in a network, which is more important when characterizing the learning dynamics of DNNs.

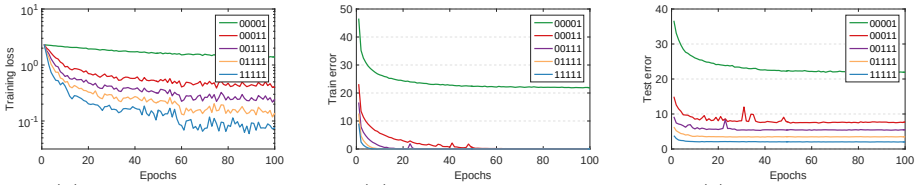
Empirical Analysis We further conduct experiments to show how the activation/gradient is affected by initialization in unnormalized DNNs (indicated as 'Plain') and batch normalized DNNs (indicated as 'BN'). We train a 20-layer MLP, with 256 neurons in each layer, for MNIST classification. The nonlinearity is ReLU. We use the full gradient descent[2], and report the results based on the best training loss among learning rates in $\{0.05, 0.1, 0.5, 1\}$. In Figure 3 (a) and (b), we observe that the magnitude of the layer input (output-gradient) of 'Plain' for random initialization [27] suffers from exponential decrease during forward pass (backward pass). The main reason for this is that the weight has a small magnitude, based on Theorem 1. This problem can be relieved by He-initialization [14], where the magnitude of the input/output-gradient is stable across layers (Figure 3 (c) and (d)). We observe that BN can well preserve the magnitude of the input/output-gradient across different layers for both initialization methods.

[2] We also perform SGD with a batch size of 1024, and further perform experiments on convolutional neural networks (CNNs) for CIFAR-10 and ImageNet. The results are shown in *SM*, in which we have the same observation as the full gradient descent.



(a) $\lambda_{max}(\Sigma_x)$ (b) $\lambda_{max}(\Sigma_{\nabla \mathbf{h}})$ (c) $\lambda_{max}(\Sigma_x)$ (d) $\lambda_{max}(\Sigma_{\nabla \mathbf{h}})$

Fig. 3. Analysis of the magnitude of the layer input (indicated by $\lambda_{max}(\Sigma_x)$) and layer output-gradient (indicated by $\lambda_{max}(\Sigma_{\nabla \mathbf{h}})$). The experiments are performed on a 20-layer MLP with 256 neurons in each layer, for MNIST classification. The results of (a)(b) are under random initialization [27], while (c)(d) He-initialization [14].



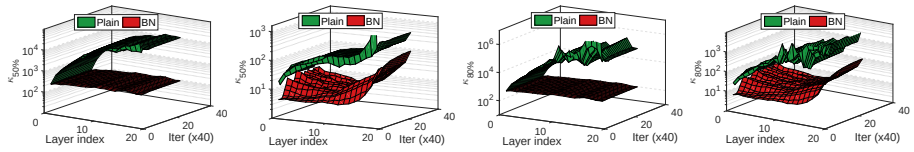
(a) Training loss (b) Training error (c) Test error

Fig. 4. Exploring the effectiveness of weight domination. We run the experiments on a 5-layer MLP with BN and the number of neuron in each layer is 256. We simulate weight domination in a given layer by blocking its weight updates. We denote '0' in the legend as the state of weight domination (the first digit represents the first layer).

Weight Domination It was shown the scale-invariant property of BN has an implicit early stopping effect on the weight matrices [2], helping to stabilize learning towards convergence. Here, we show that this layer-wise 'early stopping' sometimes results in the false impression of a local minimum, which has detrimental effects on the learning, since the network does not well learn the representation in the corresponding layer. For illustration, we provide a rough definition termed *weight domination*, with respect to a given layer.

Definition 1 *Let $\mathbf{W}_k$ and $\frac{\partial \mathcal{L}}{\partial \mathbf{W}_k}$ be the weight matrix and its gradient in layer k. If $\lambda_{max}(\frac{\partial \mathcal{L}}{\partial \mathbf{W}_k}) \ll \lambda_{max}(\mathbf{W}_k)$, where $\lambda_{max}(\cdot)$ indicates the maximum singular value of a matrix, we refer to layer k has a state of* ***weight domination****.*

Weight domination implies a smoother gradient with respect to the given layer. This is a desirable property for linear models (the distribution of the input is fixed), where the optimization objective targets to arrive the stationary points with smooth (zero) gradient. However, weight domination is not always desirable for a given layer of a DNN, since such a state of one layer is possibly caused by the increased magnitude of the weight matrix or decreased magnitude of the layer input (the non-convex optimization in Eqn. 7), not necessary driven by the optimization objective itself. Although BN ensures a stable distribution of layer inputs, a network with BN still has the possibility that the magnitude of the weight in a certain layer is significantly increased. We experimentally observe this phenomenon, as shown in the *SM*. A similar phenomenon is also observed in [47], where BN results in large updates of the corresponding weights.



(a) $\kappa_{50\%}(\Sigma_x)$ (b) $\kappa_{50\%}(\Sigma_{\nabla \mathbf{h}})$ (c) $\kappa_{80\%}(\Sigma_x)$ (d) $\kappa_{80\%}(\Sigma_{\nabla \mathbf{h}})$

Fig. 5. Analysis on the condition number of the layer input ($\kappa_p(\Sigma_x)$) and layer output-gradient ($\kappa_p(\Sigma_{\nabla \mathbf{h}})$). The experimental setups are the same as in Figure 3.

Weight domination sometimes harms the learning of the network, because this state limits its ability to learn the representation in the corresponding layer. To investigate this, we conduct experiments on a 5-layer MLP and show the results in Figure 4. We observe that the network with weight domination in certain layers, can still decrease the loss, but has degenerated performance. We also conduct experiments on CNNs for CIFAR-10 datasets, shown in the *SM*.

4.2 Improved Conditioning

One motivation behind BN is that whitening the input can improve the conditioning of the optimization [22] (*e.g.*, the Hessian will be an identity matrix under the condition that $\mathbb{E}_{\mathcal{D}}(\mathbf{x}\mathbf{x}^T) = \mathbf{I}$ for a linear model, based on Eqn. 1, and thus can accelerate training [9,21]. However, such a motivation is seldom validated by either theoretical or empirical analysis on the context of DNNs [9,38]. Furthermore, it only holds under the condition that BN is placed before the linear layer, while, in practice, BN is typically placed after the linear layer, as recommended in [22]. In this section, we will empirically explore this motivation using our layer-wise conditioning analysis for the scenario of training DNNs.

We first experimentally observe that BN not only improves the conditioning of the layer input's covariance matrix, but also improves the conditioning of the output-gradient's covariation, as shown in Figure 5. It has been shown that centered data is more likely to be well-conditioned [28,40,33,20]. This suggests that placing BN after the linear layer can improve the conditioning of the output-gradient, because centering the activation, with the gradient back-propagating through such a transformation [22], also centers the gradient.

We also observe that the unnormalized network ('Plain') has several small eigenvalues. For further exploration, we monitor the output of each neuron in each layer, and find that 'Plain' has some neurons that are not activated (zero output of ReLU) for all training examples. We refer to these neurons as *dying neurons*. We also observe that 'Plain' has some neurons that are always activated for every training example, which we refer to as *full neurons*. This observation is most obvious in the initial iterations. The number of dying/full neurons increases as the layer number increases (Please refer to *SM* for details). We conjecture that the dying neurons causes 'Plain' to have numerous small/zero eigenvalues. In contrast, batch normalized networks have no dying/full neurons, because the centering operation ensures that half the examples get activated. This further suggests that placing BN before the nonlinear activation can improve the conditioning.

5 Training Very Deep Residual Networks

Residual networks [15] have significantly relieved the difficulty of training deep networks by their introduction of the residual connection, which makes training networks with hundreds or even thousands of layers possible. However, residual networks also suffer from degenerated performance when the model is extremely deep (*e.g.*, the 1202-layer residual network has worse performance than the 110-layer one), as shown in [15]. He *et al.*[15] argued that this is from over-fitting, not optimization difficulty. Here, we show that a very deep residual network may also suffer difficulty in optimization.

We perform experiments on CIFAR-10 with residual networks, following the same experimental setup as in [15], except that we run the experiments on one GPU. We vary the network depth, ranging in $\{56, 110, 230, 1202\}$, and show the training loss in Figure 6 (a). We observe the residual networks have an increased loss in the initial iterations, which is amplified for deeper networks. Later, the training gets stuck in a state of randomly guessing (the loss stays at $\ln 10$). Although the networks can escape such a state with enough iterations, they suffer from degenerated training performance, especially if they are very deep.

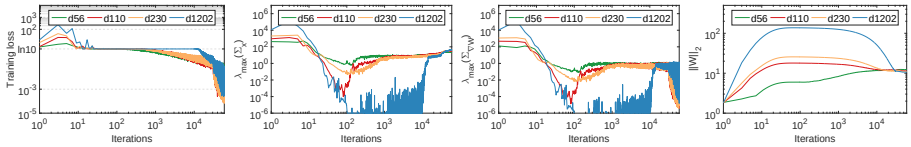
Analysis of Learning Dynamics To explore why residual networks have such a mysterious behavior, we perform the layer-wise conditioning analysis on the last linear layer (before the cross entropy loss). We monitor the maximum eigenvalue of the covariance matrix $\lambda_{\Sigma_{\mathbf{x}}}$, the maximum eigenvalue of the second moment matrix of the weight-gradient $\lambda_{\Sigma_{\frac{\partial \mathcal{L}}{\partial \mathbf{W}}}}$, and the norm of the weight ($\|\mathbf{W}\|_2$).

We observe that the initial increase in loss is mainly caused by the large magnitude of $\lambda_{\Sigma_{\mathbf{x}}}$[3] (Figure 6 (b)), which results in a large magnitude for $\lambda_{\Sigma_{\frac{\partial \mathcal{L}}{\partial \mathbf{W}}}}$ (Figure 6 (c)), and thus a large magnitude for $\|\mathbf{W}\|_2$ (Figure 6 (d)). The increased $\|\mathbf{W}\|_2$ further facilities the increase of the loss. However, the learning objective is to decrease the loss, and thus it should decrease the magnitude of $\mathbf{W}$ or $\mathbf{x}$ (based on Eqn. 7) in this case. Apparently, $\mathbf{W}$ is harder to adjust, because the landscape of its loss surface is controlled by $\mathbf{x}$, and all the values of $\mathbf{x}$ are non-negative with large magnitude. The network thus tries to decrease $\mathbf{x}$ based on the given learning objective. We experimentally find that the learnable parameters of BN have a large number of negative values, which causes the ReLUs (positioned after the residual adding operation) deactivated. Such a dynamic results in a significant reduction in the magnitude of $\lambda_{\Sigma_{\mathbf{x}}}$. The small $\mathbf{x}$ and large $\mathbf{W}$ drive the last linear layer of the network into the state of weight domination, and make the network display a random guess behavior. Although the residual network can escape such a state with enough iterations, the weight domination hinders optimization and results in degenerated training performance.

5.1 Proposed Solution

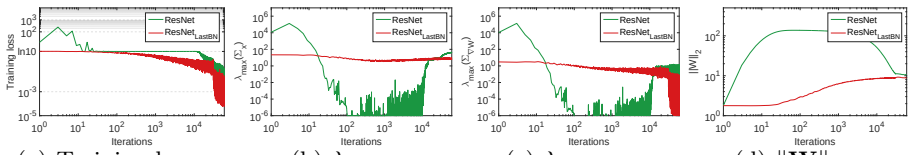
Based on the above analysis, it is essential to reduce the large magnitude of $\lambda(\Sigma_{\mathbf{x}})$. We propose a simple solution and add one BN layer before the last linear layer

[3] The large magnitude of $\lambda_{\Sigma_{\mathbf{x}}}$ is caused mainly by the addition of multiple residual connections from the previous layers with ReLU output.



(a) Training loss (b) $\lambda_{\Sigma_{\mathbf{x}}}$ (c) $\lambda_{\Sigma_{\frac{\partial \mathcal{L}}{\partial \mathbf{W}}}}$ (d) $\|\mathbf{W}\|_2$

Fig. 6. Analysis on the last linear layer in residual networks for CIFAR-10 classification. We vary the depth ranging in $\{56, 110, 230, 1202\}$ and analyze the results over the course of training. We show (a) the training loss; (b) the maximum eigenvalue of the input's covariance matrix; (c) the maximum eigenvalue of the second moment matrix of the weight-gradient; and (d) the F2-norm of the weight. Note that both the x- and y-axes are in log scale.



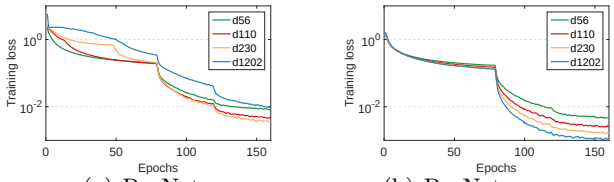
(a) Training loss (b) $\lambda_{\Sigma_{\mathbf{x}}}$ (c) $\lambda_{\Sigma_{\frac{\partial \mathcal{L}}{\partial \mathbf{W}}}}$ (d) $\|\mathbf{W}\|_2$

Fig. 7. Analysis of how ResNet$_{LastBN}$ solves the ill-conditioned problem of its last linear layer on the 1202-layer network for CIFAR-10 classification.

to normalize its input. We refer to this residual network as 'ResNet$_{LastBN}$', and the original one as 'ResNet'. We also conduct an analysis on the last linear layer of ResNet$_{LastBN}$, providing a comparison between ResNet and ResNet$_{LastBN}$ on the 1202-layer in Figure 7. We observe that ResNet$_{LastBN}$ can be steadily trained. It does not reach the state of weight domination or encounter a large magnitude of $\mathbf{x}$ in the last linear layer.

We try a similar solution where a constant is divided before the linear layer, and we find it also benefits the training. However, the main disadvantage of this solution is that the value of the constant has to be finely tuned on networks with different depths. We also try putting one BN before the average pooling, which has similar effects as putting it before the last linear layer. We note that Bjorck *et al.*[4] proposed to train a 110-layer residual network with only one BN layer, which is placed before the average pooling. They showed that this achieves good results. However, we argue that this does not hold for very deep networks. We perform an experiment on the 1202-layer residual network, and find that the model always fails in training with various hyper-parameters.

ResNet$_{LastBN}$, a simple revision of ResNet, achieves significant improvement in performance for very deep residual networks. Figure 8 (a) and (b) show the training loss of ResNet and ResNet$_{LastBN}$, respectively, on the CIFAR-10 dataset. We observe that ResNet, with a depth of 1202, appears to have degenerated training performance, especially in the initial phase. Note that, as the depth increases, ResNet obtains worse training performance in the first 80 epochs (before the learning rate is reduced), which coincides with our previous analysis. ResNet$_{LastBN}$ obtains nearly the same training loss for the networks with different depths in the first 80 epochs. Moreover, ResNet$_{LastBN}$ shows lower



(a) ResNet (b) ResNet$_{LastBN}$

Fig. 8. Training loss comparison between (a) ResNet and (b) ResNet$_{LastBN}$ with different depth on CIFAR-10. We evaluate the training loss with respect to the epochs.

Table 1. Comparison of test error (%) on CIFAR-10. The results are shown in the format of 'mean ± std' computed over five random seeds.

method	depth-56	depth-110	depth-230	depth-1202
ResNet [15]	7.52 ± 0.30	6.89 ± 0.52	7.35 ± 0.64	9.42 ± 3.10
PreResNet [16]	6.89 ± 0.09	6.25 ± 0.08	6.12 ± 0.21	6.07 ± 0.10
PreResNet$_{v1}$	6.75 ± 0.26	6.37 ± 0.24	6.32 ± 0.21	7.89 ± 0.58
ResNet$_{LastBN}$	**6.50 ± 0.22**	**6.10 ± 0.09**	**5.94 ± 0.18**	**5.68 ± 0.14**

training loss with increasing depth. Comparing Figure 8 (b) to (a), we observe that ResNet$_{LastBN}$ has better training loss than ResNet for all depths (*e.g.*, at a depth of 56, the loss of ResNet is 0.081, while for ResNet$_{LastBN}$ it is 0.043.).

Table 1 shows the test errors. We observe that ResNet$_{LastBN}$ achieves better test performance with increasing depth, while ResNet has degenerated performance. Compared to ResNet, ResNet$_{LastBN}$ has consistently improved performance over different depths. Particularly, ResNet$_{LastBN}$ reduces the absolute test error of ResNet by 1.02%, 0.79%, 1.41% and 3.74% at depths of 56, 110, 230 and 1202, respectively. Due to ResNet$_{LastBN}$'s optimization efficiency, the training performance is likely improved if we amplify the regularization of the training. Thus, we set the weight decay to 0.0002 and double the training iterations, finding that the 1202-layer ResNet$_{LastBN}$ achieves a test error of 4.79 ± 0.12. We also train a 2402-layer network. We observe that *ResNet* cannot converge, while ResNet$_{LastBN}$ achieves a test error of 5.04 ± 0.30.

We further perform the experiment on CIFAR-100 and use the same experimental setup as CIFAR-10. Table 2 shows the test errors. ResNet$_{LastBN}$ reduces the absolute test error of ResNet by 0.78%, 1.25%, 3.45% and 4.98% at depths of 56, 110, 230 and 1202, respectively. We also validate the effectiveness of ResNet$_{LastBN}$ on the large-scale ImageNet classification, with 1000 classes [8]. ResNet$_{LastBN}$ has better optimization efficiency and achieves better test performance, compared to ResNet. Please refer to the *SM* for more details.

5.2 Revisiting the Pre-activation Residual Network

We note that He *et al.*[16] tried to improve the optimization and generalization of the original residual network [15] by re-arranging the activation functions (using the pre-activation). By looking into the implementation of [16], we find that it also uses an extra BN layer before the last average pooling. It is interesting to investigate which component in [16] (*e.g.*, the pre-activation or the extra BN

Table 2. Comparison of test error (%) on CIFAR-100. The results are shown in the format of 'mean ± std', computed over five random seeds.

method	depth-56	depth-110	depth-230	depth-1202
ResNet [15]	29.60 ± 0.41	28.3 ± 1.09	29.25 ± 0.44	30.49 ± 4.44
PreResNet [16]	29.29 ± 0.44	27.58 ± 0.12	26.72 ± 0.33	26.23 ± 0.26
PreResNet_{v1}	29.60 ± 0.21	28.54 ± 0.26	27.92 ± 0.34	30.07 ± 2.04
ResNet_{LastBN}	**28.82 ± 0.38**	**27.05 ± 0.23**	**25.80 ± 0.10**	**25.51 ± 0.27**

layer) benefits the optimization behaviors, using our analysis. Here, we denote 'PreResNet' as the pre-activation residual network [16], and denote 'PreResNet_{v1}' as the PreResNet without the extra BN layer. We use the conditioning analysis on the last linear layer of the 1202-layer network (see *SM* for details). We observe that: 1) PreResNet_{v1} also gets stuck in the weight domination state with its last linear layer, even though it escapes this states faster than ResNet; 2) PreResNet, like our proposed ResNet_{LastBN}, does not suffer the ill-conditioned problem in its last linear layer. These observations suggest that the pre-activation can relieve the ill-conditioned problem to some degree, but more importantly, the extra BN layer is key to improving the optimization efficiency of PreResNet [16] for very deep networks.

We report the test errors of PreResNet and PreResNet_{v1} in Table 1 and 2. We find that 'PreResNet' generally has better test performance than PreResNet_{v1}, especially for very deep networks (*e.g.*, the 1202-layer one). This supports our arguments that the extra BN layer is the key component of PreResNet [16] for very deep networks. Interestingly, we further observe that our proposed ResNet_{LastBN} is consistently better than PreResNet [16] over different layers and datasets. This demonstrates the effectiveness of our proposed architecture. We believe that our analysis method can be further used to improve residual architectures by looking into the intermediate (inner) layers of networks.

6 Conclusion and Future Work

We proposed a layer-wise conditioning analysis to investigate the learning dynamics of DNNs. Such an analysis is theoretically derived under mild assumptions that approximately hold in practice. Based on our layer-wise conditioning analysis, we showed how batch normalization stabilizes training and improves the conditioning of the optimization problem. We further found that very deep residual networks still suffer difficulty in optimization, which is caused by the ill-conditioned state of the last linear layer. We remedied this by adding only one BN layer before the last linear layer.

We believe there are many potential applications of our method, *e.g.*, investigating the training dynamics of other normalization methods (layer normalization [2] and instance normalization [42]) and comparing them to BN. We also believe it would be interesting to analyze the training dynamics of GANs [6] using our method. We expect our method to provide new insights for analyzing and understanding training techniques for DNNs.

References

1. Ba, J., Grosse, R., Martens, J.: Distributed second-order optimization using kronecker-factored approximations. In: ICLR (2017) 2, 4, 6
2. Ba, J., Kiros, R., Hinton, G.E.: Layer normalization. CoRR **abs/1607.06450** (2016) 8, 9, 14
3. Bernacchia, A., Lengyel, M., Hennequin, G.: Exact natural gradient in deep linear networks and its application to the nonlinear case. In: NeurIPS (2018) 2, 4
4. Bjorck, J., Gomes, C., Selman, B.: Understanding batch normalization. In: NeurIPS (2018) 12
5. Bottou, L., Curtis, F.E., Nocedal, J.: Optimization methods for large-scale machine learning. Siam Review **60**(2), 223–311 (2018) 3
6. Brock, A., Donahue, J., Simonyan, K.: Large scale GAN training for high fidelity natural image synthesis. In: ICLR (2019) 15
7. Carreira-Perpinan, M., Wang, W.: Distributed optimization of deeply nested systems. In: AISTATS (2014) 2, 6
8. Deng, J., Dong, W., Socher, R., Li, L.J., Li, K., Fei-Fei, L.: ImageNet: A Large-Scale Hierarchical Image Database. In: CVPR (2009) 13
9. Desjardins, G., Simonyan, K., Pascanu, R., kavukcuoglu, k.: Natural neural networks. In: NeurIPS (2015) 10
10. Frerix, T., Möllenhoff, T., Möller, M., Cremers, D.: Proximal backpropagation. In: ICLR (2018) 2, 6, 7
11. Ghorbani, B., Krishnan, S., Xiao, Y.: An investigation into neural net optimization via hessian eigenvalue density. In: ICML (2019) 2, 4, 5
12. Glorot, X., Bengio, Y.: Understanding the difficulty of training deep feedforward neural networks. In: Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics, AISTATS 2010 (2010) 1
13. Grosse, R.B., Martens, J.: A kronecker-factored approximate fisher matrix for convolution layers. In: ICML (2016) 3
14. He, K., Zhang, X., Ren, S., Sun, J.: Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In: ICCV (2015) 1, 6, 8, 9
15. He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: CVPR (2016) 1, 2, 7, 11, 13, 14
16. He, K., Zhang, X., Ren, S., Sun, J.: Identity mappings in deep residual networks. In: ECCV (2016) 2, 13, 14
17. Hinton, G.E., Salakhutdinov, R.R.: Reducing the dimensionality of data with neural networks. Science **313**, 504–507 (2006) 1
18. Hoffer, E., Banner, R., Golan, I., Soudry, D.: Norm matters: efficient and accurate normalization schemes in deep networks. NeurIPS (2018) 8
19. Huang, G., Liu, Z., Weinberger, K.Q.: Densely connected convolutional networks. In: CVPR (2017) 7
20. Huang, L., Liu, X., Liu, Y., Lang, B., Tao, D.: Centered weight normalization in accelerating training of deep neural networks. In: ICCV (2017) 10
21. Huang, L., Yang, D., Lang, B., Deng, J.: Decorrelated batch normalization. In: CVPR (2018) 10
22. Ioffe, S., Szegedy, C.: Batch normalization: Accelerating deep network training by reducing internal covariate shift. In: ICML (2015) 1, 2, 5, 6, 7, 8, 10
23. Karakida, R., Akaho, S., Amari, S.i.: Universal statistics of fisher information in deep neural networks: Mean field approach. In: AISTATS (2019) 5

24. Kingma, D.P., Ba, J.: Adam: A method for stochastic optimization. CoRR **abs/1412.6980** (2014) 1
25. Kohler, J., Daneshmand, H., Lucchi, A., Zhou, M., Neymeyr, K., Hofmann, T.: Towards a theoretical understanding of batch normalization. arXiv preprint arXiv:1805.10694 (2018) 1
26. LeCun, Y., Bengio, Y., Hinton, G.E.: Deep learning. Nature **521**, 436–444 (2015) 1
27. LeCun, Y., Bottou, L., Orr, G.B., Müller, K.R.: Effiicient backprop. In: Neural Networks: Tricks of the Trade. pp. 9–50 (1998) 1, 3, 5, 8, 9
28. LeCun, Y., Kanter, I., Solla, S.A.: Second order properties of error surfaces: Learning time and generalization. In: NeurIPS (1990) 1, 3, 10
29. Martens, J.: Deep learning via hessian-free optimization. In: ICML. pp. 735–742 (2010) 3
30. Martens, J.: New perspectives on the natural gradient method. CoRR **abs/1412.1193** (2014) 3, 4
31. Martens, J., Grosse, R.: Optimizing neural networks with kronecker-factored approximate curvature. In: ICML (2015) 2, 4, 6
32. Martens, J., Sutskever, I., Swersky, K.: Estimating the hessian by back-propagating curvature. In: ICML (2012) 2
33. Montavon, G., Müller, K.R.: Deep Boltzmann Machines and the Centering Trick, LNCS, vol. 7700 (2012) 10
34. Papyan, V.: The full spectrum of deep net hessians at scale: Dynamics with sample size. CoRR **abs/1811.07062** (2018) 2, 4
35. Pascanu, R., Bengio, Y.: Revisiting natural gradient for deep networks. In: ICLR (2014) 3
36. Roux, N.L., Manzagol, P., Bengio, Y.: Topmoumoute online natural gradient algorithm. In: NeurIPS. pp. 849–856 (2007) 3
37. Sagun, L., Evci, U., Güney, V.U., Dauphin, Y.N., Bottou, L.: Empirical analysis of the hessian of over-parametrized neural networks. CoRR **abs/1706.04454** (2017) 2, 3
38. Santurkar, S., Tsipras, D., Ilyas, A., Madry, A.: How does batch normalization help optimization? In: NeurIPS (2018) 1, 5, 10
39. Saxe, A.M., McClelland, J.L., Ganguli, S.: Exact solutions to the nonlinear dynamics of learning in deep linear neural networks. In: ICLR (2014) 1
40. Schraudolph, N.N.: Accelerated gradient descent by factor-centering decomposition. Tech. rep. (1998) 10
41. Sun, K., Nielsen, F.: Relative Fisher information and natural gradient for learning large modular models. In: ICML (2017) 2, 4
42. Ulyanov, D., Vedaldi, A., Lempitsky, V.S.: Instance normalization: The missing ingredient for fast stylization. CoRR **abs/1607.08022** (2016) 14
43. Wei, M., Stokes, J., Schwab, D.J.: Mean-field analysis of batch normalization. arXiv:1903.02606 (2019) 5
44. Wiesler, S., Ney, H.: A convergence analysis of log-linear training. In: NeurIPS (2011) 1, 3, 4
45. Wu, S., Li, G., Deng, L., Liu, L., Xie, Y., Shi, L.: L1-norm batch normalization for efficient training of deep neural networks. CoRR (2018) 8
46. Wu, Y., He, K.: Group normalization. In: ECCV (2018) 1
47. Yang, G., Pennington, J., Rao, V., Sohl-Dickstein, J., Schoenholz, S.S.: A mean field theory of batch normalization. In: ICLR (2019) 8, 10
48. Zagoruyko, S., Komodakis, N.: Wide residual networks. In: BMVC (2016) 7
49. Zeiler, M.D.: ADADELTA: an adaptive learning rate method. CoRR **abs/1212.5701** (2012) 1

50. Zhang, H., Chen, W., Liu, T.Y.: On the local hessian in back-propagation. In: NeurIPS (2018) 2, 6, 7